CityPulse Team Execution Guide

Follow your part in order. Every step is small and ends with a **Done when** check. Do not start the next step until the check passes. Part 5 shows how all four parts connect.

Person	Role	Section
Muhammad Fasih	Audio lead	Part 1
Bushra Zafar	Image lead	Part 2
Abdul Hayy Khan	Data and ML lead	Part 3
Zainab Siddiq	Full-stack lead	Part 4

0. Shared Setup (everyone)

1. **One GitHub repo** named citypulse, using the folder structure from the proposal (Section 15). Each person works on their own branch (audio, image, data-ml, fullstack) and pushes only to that branch. When a phase is complete, the branches are merged into a **dev** branch. After finalization, dev is merged into main. Nobody pushes directly to main. The repo owner adds the other three as collaborators.
2. **One conda environment** (Python 3.10+). Shared requirements.txt: pandas, numpy, scikit-learn, scipy, matplotlib, seaborn, joblib, librosa, soundfile, torch, torchvision, pillow, fastapi, uvicorn, python-multipart, osmnx, networkx, requests.
3. **Never commit raw datasets.** Add data/raw/ to .gitignore. Do commit small results: metric tables (CSV) and figures (PNG).
4. **Daily sync:** what I finished, what I do today, what blocks me. Post your metric table in the group chat after every model.
5. **The interface contract in Part 5 is fixed.** If you must change a function name or file format, tell the others first.

Done when: everyone can clone the repo, activate the environment, and run python -c 'import librosa, sklearn, fastapi'.

Part 1: Fasih (Audio Lead)

Your output: a function predict_audio(path), a saved model, a metrics table, a degraded-audio result, and a cross-dataset result.

Step 1. Download the audio data

- UrbanSound8K: Zenodo record 1203745 (or Kaggle). You need the audio/fold1 to fold10 folders and metadata/UrbanSound8K.csv.
- ESC-50: github.com/karolpiczak/ESC-50. You need audio/ and meta/esc50.csv.

Done when: you can load and play one clip from each dataset.

Step 2. Build one label table

Create labels.csv with columns: path, source (us8k or esc50), fold, original_class, app_class. Use this mapping:

App class	UrbanSound8K	ESC-50
traffic_horn	car_horn, engine_idling	car horn, engine
construction	drilling, jackhammer	chainsaw, hand saw
siren	siren	siren
music_loudspeaker	street_music	none
hvac	air_conditioner	none

Skip gun_shot, dog_bark, and children_playing. **Use ESC-50 only as a test set** (Step 7), never for training.

Done when: you print the clip count per app class and per source.

Step 3. Fixed length and features

Every clip becomes 4 seconds at 22,050 Hz, mono. Shorter clips are padded, longer ones are cropped. The browser records 5 seconds, so the app will crop the same way.

```
y, sr = librosa.load(path, sr=22050, mono=True)
y = librosa.util.fix_length(y, size=sr * 4)
mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=64)
logmel = librosa.power_to_db(mel, ref=np.max)
feat = np.concatenate([logmel.mean(axis=1), logmel.std(axis=1)])  # 128
numbers
```

Save all features to data/processed/audio_features.npy with a matching labels file.

Done when: the feature matrix has shape (number of clips, 128) and contains no NaN.

Step 4. Relative intensity

ref=np.max in Step 3 removes loudness, so compute intensity separately from the raw clip:

```
rms = np.sqrt(np.mean(y ** 2))
db = 20 * np.log10(rms + 1e-9)
```

Rescale db to a 0 to 1 score using the 5th and 95th percentile of the training clips. This is **relative** loudness only, never real decibels.

Done when: quiet air-conditioner clips score low and jackhammer clips score high on average.

Step 5. Train SVM and Random Forest

- Use UrbanSound8K's **predefined folds**. Train on folds 1 to 9 and test on fold 10, or run 10-fold cross-validation grouped by fold. Never reshuffle randomly (it leaks clips from the same recording).
- Build a Pipeline: StandardScaler, then SVC(kernel='rbf', probability=True). GridSearchCV over C in [1, 10, 100] and gamma in ['scale', 0.01, 0.001].
- Train RandomForestClassifier with 200 and 300 trees and compare.
- Save accuracy, the classification report, and a confusion matrix image.

Done when: you have an SVM vs Random Forest table with per-class F1.

Step 6. Simulate cheap phone microphones

Write degrade(y, sr) that applies: a low-pass filter around 4 kHz (scipy.signal.butter), added background noise at 10 to 20 dB signal-to-noise ratio, and mild clipping. Then:

1. Test the clean-trained model on degraded test clips.
2. Retrain with degraded copies added to the training set and test again.

In the report call this **simulated degradation**, not real phone performance.

Done when: you have a clean vs degraded accuracy table.

Step 7. Cross-dataset test

Train on UrbanSound8K only. Test on the ESC-50 clips that match your classes (car horn, engine, siren, chainsaw, hand saw). Expect a drop. That drop is a finding, so report it honestly, together with the ESC-50 caveat that some source clips were band-limited.

Done when: you have a table: in-dataset accuracy vs cross-dataset accuracy.

Step 8. Optional: small CNN (only if ahead)

Train a small CNN on the 64-band log-mel image and use its second-last layer as an embedding for the SVM. If you are behind schedule, skip this step. The feature-based SVM and Random Forest already satisfy the lab concepts.

Step 9. Package for Zainab

- Save models/audio_model.joblib.
- Write src/models/audio.py with predict_audio(path) returning {type, confidence, relative_intensity}. It must accept a wav file; ask Zainab to convert browser webm to wav first.
- Copy 10 test clips into demo_samples/audio/ (from fold 10 and ESC-50), covering every class.

Done when: Zainab calls predict_audio on a demo clip and gets the right answer.

Report material you hand over: your metric tables, the confusion matrix, the degradation and cross-dataset tables, and a limitations paragraph.

Part 2: Bushra (Image Lead)

Your output: a function predict_image(path), a saved model, a metrics table, and a flood cross-set result.

Step 1. Download RDD2022

Start this immediately because it is about 13 GB. Go to github.com/sekilab/RoadDamageDetector and follow the link to the Figshare release. **Use only the India subset** (about 9,665 images).

Done when: you can list India images and their XML files.

Step 2. Inspect the annotations

The XML files are Pascal VOC. Parse them with xml.etree.ElementTree. Each object has a name (D00, D10, D20, D40) and a box (xmin, ymin, xmax, ymax). D40 is pothole. D00, D10, and D20 are cracks.

Done when: you print counts per damage class and the number of XML files with zero objects.

Step 3. Build the classification dataset

The app receives a whole photo, so prefer **image-level labels**:

- pothole: image has at least one D40
- crack: image has cracks but no D40
- normal: image has no annotations

If fewer than a few hundred images have no annotations, switch to **patches**: crop pothole and crack boxes (skip boxes smaller than 32 pixels) and cut random normal patches that overlap no box. Cap each class at about 1,500 samples to save time.

Split by image, never by patch (GroupShuffleSplit with the image id), 70/15/15, so crops of one photo never land in both train and test.

Done when: you have train, validation, and test folders with class counts printed.

Step 4. Get flood images

- AlleyFloodNet: find the dataset link in the paper (preprints.org, manuscript 202504.0154). It is a preprint, so check the license.
- Floodwater-on-roadways dataset: arXiv 1909.00125, via the authors' link.

Put them in flood/ and non_flood/ folders. Non-flood images can be added to the normal class.

Fallback: if neither dataset can be downloaded, drop the flood class from the image classifier and tell the team. Flooding is still covered by the 311 flooding complaints and the rainfall regression.

Step 5. Extract embeddings

Use a pretrained MobileNetV2 as a fixed feature extractor: torchvision.models.mobilenet_v2(weights='DEFAULT'), drop the classifier, keep the pooled 1,280-number output. Resize to 224 by 224 and normalize with ImageNet mean and standard deviation. Run on CPU in batches and save X_train.npy, X_val.npy, X_test.npy with labels.

Done when: the arrays load and their row counts match the labels.

Step 6. Train SVM and Random Forest

Use SVC(probability=True) with GridSearchCV over C and gamma, and RandomForestClassifier. Set class_weight='balanced' because pothole and normal counts will differ. Save accuracy, per-class precision, recall, F1, and the confusion matrix.

Done when: you have an SVM vs Random Forest table.

Step 7. Flood cross-set test

Train the flood vs non-flood decision on one dataset and test on the other. Because the datasets are small, compute a bootstrap confidence interval for the test accuracy and report it. Do not over-claim.

Step 8. Package for Zainab

- Save models/image_model.joblib.
- Write src/models/image.py with predict_image(path) returning {type, confidence}.
- Copy 10 test photos into demo_samples/images/, covering every class.

Done when: Zainab calls predict_image on a demo photo and gets a sensible answer.

Step 9. Optional (only if ahead)

Try horizontal flips and brightness changes on the training images and check whether the F1 score improves.

Report material you hand over: dataset counts, metric tables, confusion matrix, flood cross-set result with confidence interval, and the limitation that flood data is small.

Part 3: Abdul Hayy (Data and ML Lead)

You own the NYC 311 data, which feeds Labs 2, 3, 10, and 11. Your output is a set of files and functions listed in Part 5.

Step 1. Explore and pull the data

The dataset is NYC 311 Service Requests from 2020 to Present, id erm2-nwe9, on data.cityofnewyork.us (Socrata API). First list the complaint types so you know the real names:

```
import requests
url = 'https://data.cityofnewyork.us/resource/erm2-nwe9.json'
r = requests.get(url, params={'$select': 'complaint_type, count(*)',
'$group': 'complaint_type', '$order': 'count(*) DESC', '$limit': 60})
print(r.json())
```

Then pick **one borough and one recent 12-month window** and pull the rows you need, 50,000 at a time using $limit and $offset, keeping only rows with latitude and longitude not null. Save the raw CSV in data/raw/ (do not commit it).

Identify three groups by looking at the real complaint_type and descriptor values: **noise** (types starting with Noise), **road condition** (pothole-type street condition), and **flooding** (street flooding or sewer related). Write down the exact names you found.

Done when: you have a raw CSV and a short table of the exact complaint types and descriptors chosen for each group.

Step 2. Clean the data, Lab 1

- Drop rows with missing coordinates and points outside the study area.
- Parse created_date and closed_date; add hour, weekday, month, and time-to-close.
- One-hot encode complaint type, descriptor group, and channel.
- Scale numeric features with StandardScaler.

Save data/processed/complaints_clean.csv.

Done when: you have a before/after row-count table and a missing-value table for the report.

Step 3. Hotspot zones with K-Means, Lab 10

1. Convert latitude and longitude to meters: x = lon * 111320 * cos(lat), y = lat * 110540.
2. Run K-Means for K = 2 to 15 and plot inertia (the elbow). Compute silhouette on a random sample of 10,000 points.
3. Choose K, then save zones.json with zone id, centre, count, and dominant complaint group. Add zone_id to every complaint.

Done when: you have an elbow plot, a silhouette score, and a zone map picture.

Step 4. Duplicate merging with hierarchical clustering, Lab 11

Hierarchical clustering is slow on millions of rows, so work **per day and per 500 m grid cell** (blocking), and cluster inside each block.

- Features: x in meters, y in meters, and time scaled so that about 1 hour counts like about 50 m (tune this).
- AgglomerativeClustering(n_clusters=None, distance_threshold=T, linkage='average'). Choose T by plotting a dendrogram (scipy.cluster.hierarchy) on a sample of about 200 points.
- **Weak ground truth:** requests with the same address and complaint type within 24 hours. Compute ARI against it, and the cophenetic correlation.
- Save incidents.csv with incident id, centre latitude and longitude, report count, and group.

Done when: you have a dendrogram, the ARI, and incidents.csv.

Step 5. Trust scorer with logistic regression, Lab 3

1. Print value_counts of resolution_description (top 50). Manually map each to **valid** (an agency acted or found something) or **invalid** (unable to locate, no evidence, duplicate). Write the mapping down exactly. This is a **proxy label**.
2. Features: complaint type, descriptor group, hour, weekday, channel, location type, number of nearby complaints.
3. LogisticRegression(class_weight='balanced') with a **time-based split** (train on earlier months, test on later ones).
4. Report precision, recall, ROC and PR curves, and a confusion matrix. Save trust_model.joblib.

Done when: score_trust(features) returns a number between 0 and 1.

Step 6. Regression with rainfall, Lab 2

Get daily rainfall from Open-Meteo for the borough centre: archive-api.open-meteo.com/v1/archive with latitude, longitude, start_date, end_date, hourly=precipitation. Sum to daily totals.

- **Flood model:** daily flooding-complaint count from rain today, rain yesterday, rain two days ago, and the 3-day sum. LinearRegression, time-based split, report MSE, RMSE, R-squared, and plots of predicted vs actual and residuals.
- **Noise model:** noise-complaint volume from hour and weekday (one-hot) and borough.

Save predict_flood_risk(rain_mm) and predict_noise(hour, weekday).

Done when: both models have metrics and plots.

Step 7. Optional baselines (only if the core is done)

Naive Bayes (Lab 9), Decision Tree (Lab 4), and KNN (Lab 8) predicting the complaint group from metadata, compared in one table.

Step 8. Hand over to Zainab

Place complaints_clean.csv, incidents.csv, zones.json, trust_model.joblib, and the two regression functions where Zainab expects them. Give her the exact column names.

Fairness note for the report: normalize zone rankings by how many reports each area typically submits, because complaint data over-represents some neighbourhoods.

Part 4: Zainab (Full-Stack Lead)

You build the system that everyone else plugs into. Work with stubs first so nobody waits.

Step 1. FastAPI skeleton with fake data

Create backend/main.py with these endpoints returning hard-coded JSON that matches the shared schema: POST /report, GET /incidents, GET /zones, POST /predict, POST /route, GET /admin. Add CORS middleware. Run uvicorn main:app --reload.

Done when: http://localhost:8000/docs lists all six endpoints and each returns JSON.

Step 2. Frontend skeleton

Create frontend/index.html, style.css, app.js. Load Leaflet and Chart.js from a CDN. Show a map centred on the study area. Fetch /incidents and draw markers coloured by type.

Done when: the map shows the fake incidents from the API.

Step 3. Database

Use SQLite. At startup, load Abdul Hayy's complaints_clean.csv and incidents.csv with pandas to_sql. Add a reports table for demo submissions.

Done when: /incidents returns real rows once Abdul Hayy's files exist (until then, keep the stubs).

Step 4. Report endpoint

POST /report receives a file, latitude, longitude, and modality. Save the upload to a temporary file. For audio, convert browser webm to 22,050 Hz mono wav with ffmpeg (subprocess) before calling predict_audio. For images, call predict_image directly. Return type, confidence, intensity, and trust score.

Swap each stub for the real function **as soon as Fasih or Bushra delivers it**.

Done when: uploading a demo clip and a demo photo returns real predictions.

Step 5. Demo mode and live recording

- **Demo mode (required):** a dropdown of files from demo_samples/, a click on the map to choose the location, a Submit button, and a result card. This is what you will use in the live demo.
- **Live mic (optional):** navigator.mediaDevices.getUserMedia({ audio: { echoCancellation: false, noiseSuppression: false, autoGainControl: false } }), record 5 seconds with MediaRecorder, upload. The browser needs localhost or HTTPS for the microphone.

Done when: a demo submission drops a new pin on the map with a result card.

Step 6. Zones and livability score

Draw zones from zones.json as circles or polygons. Compute a livability score per zone as a weighted sum of normalized noise, road, and flood counts (start with equal weights), shown as a colour scale with a legend. Add an explainability panel listing the counts by

type and the feature importance plot Bushra and Fasih export.

Step 7. Prediction controls

A rain slider calls POST /predict with predict_flood_risk and recolors flood risk on the zones. A time-of-day slider calls predict_noise. Add a **report-density overlay** that marks low-data areas so they are never shown as safe.

Step 8. Calm and Safe Route

1. With osmnx build the drive network once around the study area (graph_from_point with about 3,000 m) and cache it with save_graphml.
2. Give every edge a weight of length multiplied by (1 + penalty). The penalty is high when the edge is within about 100 m of an incident or inside a noisy zone (use a BallTree or KD-tree on incident coordinates).
3. POST /route snaps the start and end to the nearest nodes and uses networkx shortest_path with that weight. Return the path coordinates.
4. Draw the normal route and the safe route in two colours.

Done when: adding a new incident on the normal route makes the safe route detour.

Step 9. Admin priority view

GET /admin ranks incidents by report count multiplied by a severity weight per type. Show it as a sorted table.

Step 10. Polish and package

Write a README.md with setup commands, add a run script, capture screenshots for the report, and record the demo video with the script in Part 5.

Part 5: Complete Project Flow

5.1 How the pieces connect

1. Fasih and Bushra train the audio and image classifiers on public datasets and publish two Python functions.
2. Abdul Hayy turns real public NYC 311 complaints into clean data, hotspot zones, merged incidents, a trust scorer, and forecasts.

3. Zainab's FastAPI backend loads everything, exposes it through six endpoints, and the vanilla HTML/CSS/JS map shows it.
4. In demo mode, a held-out public clip or photo is submitted at a clicked map location. The backend classifies it, trust-scores it, adds it as a pin, and the route planner avoids it.

5.2 Interface contract (fixed)

Provider	What	Format
Fasih	src/models/audio.py: predict_audio(wav_path)	returns {type, confidence, relative_intensity}
Bushra	src/models/image.py: predict_image(image_path)	returns {type, confidence}
Abdul Hayy	data/processed/complaints_clean.csv	one row per complaint: id, lat, lon, timestamp, group, descriptor, zone_id
Abdul Hayy	data/processed/incidents.csv	incident_id, lat, lon, count, group
Abdul Hayy	data/processed/zones.json	zone id, centre, count, dominant group
Abdul Hayy	score_trust(features), predict_flood_risk(rain_mm), predict_noise(hour, weekday)	each returns a number
Zainab	the six FastAPI endpoints	JSON using the shared report schema

5.3 Work order

Stage	Fasih	Bushra	Abdul Hayy	Zainab
1. Setup and data access	Setup; download audio data	Setup; start RDD2022 download; find flood data	Setup; explore 311 complaint types	Setup; FastAPI stubs; frontend skeleton

Stage	Fasih	Bushra	Abdul Hayy	Zainab
2. Data preparation	Label table; features and intensity	Inspect XML; build image dataset	Pull and clean 311 data	Database loading; report endpoint stub
3. First models	Train SVM and Random Forest	Embeddings; train SVM and Random Forest	K-Means zones	Demo mode UI
4. Evaluation	Confusion matrix; degradation test; cross-dataset test	Evaluate; flood cross-set test with confidence intervals	Duplicate merging; trust scorer	Wire predict_audio and predict_image
5. Packaging	Package model; demo clips	Package model; demo photos	Regression models; hand over files	Zones and livability score
6. Decision point	Decide together what to drop; optional CNN	Optional augmentation	Optional baselines	Prediction controls
7. Advanced features	Help with report tables	Help with report tables	Fix data bugs	Route planner; admin view
8. Final metrics and code freeze	Final metrics	Final metrics	Final metrics	README; screenshots
9. Report and demo	Report sections; review	Report sections; review	Report sections; review	Review report; record demo video; rehearse and fix

Stages show the order of work only. They are not tied to dates.

5.4 Checkpoints

- **First checkpoint:** all four stubs run end to end with fake data.
- **Second checkpoint:** real models and real data are plugged in.
- **Decision point:** decide what to cut. Optional work stops if any core item is late.

- **Final checkpoint:** code freeze. After this only bug fixes.

5.5 Who writes which report section

Report section	Owner
Audio data, models, results	Fasih
Image data, models, results	Bushra
311 data, clustering, trust, regression, ethics	Abdul Hayy
Architecture diagram, setup instructions, demo	Zainab
Introduction, lab concept table, limitations	All, reviewed together at the end

5.6 Demo script (rehearse twice)

1. Open the dashboard and show the real hotspot zones built from public 311 complaints.
2. In demo mode, submit a held-out jackhammer clip at a map location. A construction pin appears with a confidence and relative-intensity reading.
3. Submit a held-out pothole photo. A second pin appears.
4. Submit several reports near one spot and show them merging into one incident.
5. Move the rain slider and show the elevated flood-risk zones.
6. Click Plan route and show the safe route detouring around the new hazards.
7. Close by stating plainly what the data is and is not: public data, not Karachi measurements.

5.7 If you fall behind

- Flood images unavailable: drop the flood class and keep pothole, crack, and normal.
- CNN not needed: feature-based SVM and Random Forest are enough.
- Live microphone not working: use demo mode only.
- Routing too slow: show the safe route on a smaller area.
- Baselines and the Karachi view are cut first.

5.8 Final checklist

- [ ] Both classifiers saved with metric tables and confusion matrices

- [ ] Cross-dataset and degradation results reported honestly
- [ ] K-Means zones, dendrogram, trust scorer, and regression results ready
- [ ] All six endpoints return real data
- [ ] Demo mode works from a clean start
- [ ] README with setup steps, 3 to 5 page report, demo video
- [ ] Every proxy label, weak label, and limitation stated in the report